

互動式電腦圖學期末專題

即時煙霧模擬與渲染

蔡平樂 b11201024

許佑威 b11901089

June 7, 2026

1 簡介

本專題實作了一套完全在瀏覽器中執行、基於 WebGL 的即時體積煙霧模擬系統。我們以 PavelDoGreat 的開源 2D 流體模擬 [1] 為起點，並從以下幾個方向進行擴充：

1. 將 2D 求解器擴展為完整的 3D 速度、密度與溫度體積場，並以 2D texture atlas 儲存，藉此繞過 WebGL 不支援原生 3D texture 的限制。
2. 溫度場驅動浮力，並透過 vorticity confinement 恢復因數值耗散而流失的小尺度漩渦細節。
3. 體積場以逐像素 ray marching 方式渲染，並加入軟陰影與基於溫度的色彩漸層。
4. 場景中整合了 3D 實體幾何，包含程序化生成的方塊以及從 GLB 檔案載入的網格模型。
5. 新增 depth pre-pass，記錄實體表面到攝影機的距離，使 ray march 能在不透明邊界正確終止。
6. 加入第一人稱攝影機與拋射物系統，讓使用者能探索場景，並在碰撞時觸發煙霧噴發。

所有模擬步驟皆以 GLSL fragment shader 在全螢幕四邊形上執行，並對浮點數 FBO 進行讀寫。simulation.js 中的主迴圈透過 requestAnimationFrame 驅動動畫，並將 Δt 限制在 $\frac{1}{60}$ 秒以維持求解器的穩定性。

2 物理模擬

我們以 [1] 的程式碼庫為起點，並重寫所有 shader 使其作用於 3D 體積場而非 2D canvas。

2.1 以 2D Texture Atlas 儲存 3D 體積場

由於 WebGL 1.0 並未提供 3D texture，我們將 $64 \times 64 \times 64$ 的網格以 8×8 排列方式平鋪 64 個 Z 切片 ($64 \times 8 = 512$)，打包進單一張 512×512 的 texture 中。shaders.js 中定義了三個會被加入到每個 shader 開頭的 GLSL 輔助函式：encodeUVW 將 3D 座標映射到 atlas 上的 texel，decodeUVW 則進行反向映射，sampleVolume 則藉由 mix 混合相鄰兩張 Z 切片來提供 trilinear 插值。

2.2 每幀求解步驟

simulation.js 中的 step(dt) 在每一幀依序執行以下步驟。

浮力 高溫的煙霧上升、高密度的煙霧下沉 (passes/buoyancy.js)：

$$v_y += \Delta t (B \cdot T - W \cdot |\rho|), \quad B = 0.1, \quad W = 0.001$$

Vorticity confinement 利用六鄰近點有限差分計算渦度 $\omega = \nabla \times \mathbf{v}$ ，並注入回復力：

$$\mathbf{f} = \varepsilon (\hat{\eta} \times \omega), \quad \hat{\eta} = \text{normalize}(\nabla |\omega|), \quad \varepsilon = 30$$

壓力投影 不可壓縮性透過三個子步驟強制實現：(i) 以六鄰近點差分搭配 no-slip 邊界條件計算散度 $\nabla \cdot \mathbf{v}$ ；(ii) 以前一幀壓力的 0.8 倍作為初始值進行 25 次 Jacobi 迭代：

$$p = \frac{1}{6}(p_L + p_R + p_B + p_T + p_F + p_K - \text{div});$$

(iii) 速度修正 $\mathbf{v} -= \frac{1}{2} \nabla p_0$

Semi-Lagrangian 平流 速度、密度與溫度場皆各自進行回溯追蹤並重新取樣 (passes/advection.js)：

$$\mathbf{uvw}_{\text{prev}} = \mathbf{uvw} - \Delta t \cdot \mathbf{v}(\mathbf{uvw}), \quad q^{n+1} = q^n(\mathbf{uvw}_{\text{prev}}) / (1 + d \Delta t)$$

2.3 煙霧發射器

煙霧透過高斯 splat 注入 (passes/splat.js)： $q(\mathbf{uvw}) += \mathbf{c} \cdot \exp(-|\mathbf{uvw} - \mathbf{p}|^2/r)$ 。每個發射器皆由 EmitterScheduler (passes/emitter.js) 管理，支援 startTime/endTime 時間窗、可選的逐幀 trajectory callback 用於覆寫位置或強度，以及會立即填滿周遭區域的初始 burst (半徑放大 4 倍、溫度放大 2 倍)。

3 加入 3D 模型

3.1 載入 GLB 檔案

model.js 中的 `loadGLBModel` 實作了一個獨立的二進位 glTF 2.0 解析器。它先讀取 GLB 標頭以定位 JSON chunk 與 binary buffer chunk，接著重建場景圖。節點的變換會從 TRS（平移、四元數旋轉、縮放）被分解為 column-major 的 4×4 矩陣，並依 $M = TRS$ 相乘。場景圖則透過 `_traverseNode` 遞迴走訪，由父節點向子節點累積 world matrix。

Accessor 的資料透過 `_readAccessor` 讀取，可處理 FLOAT32、UINT16、UINT32、UINT8 等元件型別的緊密排列與交錯排列的 buffer view。材質顏色若存在 `pbrMetallicRoughness` 的 `base-colour factor` 則由其讀取，否則使用預設的灰藍色 (0.65, 0.70, 0.80)。Bounding box 由 POSITION accessor 的 min/max 欄位推算，藉此計算出能符合指定目標大小的均勻縮放比例。

3.2 程序化場景幾何

`createSceneGeometry()` 以方塊基本圖元組成靜態場景：

物件	尺寸 (寬 × 高 × 深)	顏色 (RGB)
地板	$16 \times 0.1 \times 16$	深灰色 (0.18, 0.20, 0.24)
紅色方塊	$2 \times 2 \times 2$	磚紅色 (0.84, 0.35, 0.22)
青色方塊	$1.5 \times 1.5 \times 1.5$	青色 (0.28, 0.78, 0.82)
黃色方塊	$1 \times 1 \times 1$	沙黃色 (0.75, 0.72, 0.36)
前/後牆	$8 \times 4 \times 0.2$	石板灰 (0.32, 0.35, 0.38)
左/右牆	$0.2 \times 4 \times 16$	石板灰 (0.32, 0.35, 0.38)

每個方塊也會在 `_colliders` 中註冊一筆 AABB，用於拋射物的碰撞偵測（見第 6 節）。

3.3 GPU 上傳與 VAO 管理

每個圖元——無論來自 GLB 或是程序化生成——都透過 `_buildPrimitive` 上傳至 GPU：分別為位置與法向量建立獨立的 VBO、為索引建立 IBO，並使用 VAO (WebGL 2) 封裝所有屬性狀態，避免後續的 draw call 互相污染綁定狀態。索引緩衝區依索引陣列的型別使用 UNSIGNED_SHORT 或 UNSIGNED_INT。沒有法向量的圖元則被賦予固定的向上向量 (0, 1, 0)。

4 渲染與著色

4.1 3D 模型著色：Blinn-Phong

實體幾何由 model.js 中的 `_MODEL_FRAG` shader 渲染。主光源方向固定為 $(0.4, 0.8, 0.45)$ (在 shader 中正規化)，攝影機位置則每幀透過 `uEye` uniform 傳入：

$$I = \mathbf{c}_{\text{base}} \underbrace{(\max(\mathbf{N} \cdot \mathbf{L}, 0) \cdot 0.7 + 0.3)}_{\text{漫射 + 環境光}} + \underbrace{\max(\mathbf{N} \cdot \mathbf{H}, 0)^{48} \cdot 0.4}_{\text{鏡面反射}}$$

其中 $\mathbf{H} = \text{normalize}(\mathbf{L} + \mathbf{V})$ 。常數 0.3 提供柔和的環境光下限，指數 48 則產生適度集中的鏡面反射高光。法向量則透過 model matrix 左上角的 3×3 子矩陣轉換到 world space。

4.2 煙霧著色：體積 Ray Marching

煙霧體積由 `passes/render.js` 中的 `drawRayMarch` 進行渲染。

攝影機與光線建構 在執行時依據 `yaw` 與 `pitch` 角度建構出 FPS 攝影機基底 $(\hat{F}, \hat{R}, \hat{U})$ ：

$$\hat{d} = \text{normalize}\left(\hat{F} + \text{ndc}_x \cdot \text{aspect} \cdot \tan \frac{\text{FOV}}{2} \cdot \hat{R} + \text{ndc}_y \cdot \tan \frac{\text{FOV}}{2} \cdot \hat{U}\right)$$

視野角為 60° ，長寬比則每幀依 canvas 大小計算。

光線與方塊相交 使用 slab method 計算光線與體積 AABB (半徑 5.0，中心 $(0, -2, 12)$) 的交點，得到 t_{start} 與 t_{end} 。

步進累積迴圈 區間 $[t_{\text{start}}, t_{\text{end}}]$ 被均分為 64 個步進 (`MAX_STEPS`)。在每個取樣位置 $\mathbf{p}$ ：

1. 體積座標 UVW 為 $(\mathbf{p} - \mathbf{c}_{\text{vol}})/(2r) + 0.5$ 。密度 $\rho < 0.001$ 的取樣點將被跳過。
2. 軟陰影 朝光源方向以步長 0.12 march 三步，累積的密度會衰減光照強度： $\ell = \exp(-\sigma_{\text{sh}} \cdot \text{DENSITY_SCALE} \cdot \text{ABSORPTION} \cdot \text{SHADOW_STEP})$ 。
3. 溫度色調 取樣到的溫度會在冷色調的藍白色 (0.85, 0.90, 1.00) 與暖色調的橙色 (1.00, 0.55, 0.10) 之間進行插值： $\mathbf{c}_{\text{tint}} = \text{mix}(\text{cool}, \text{warm}, \text{clamp}(T \cdot 0.5, 0, 1))$ 。
4. 由前向後合成：

$$\alpha = 1 - e^{-\rho \Delta s \cdot \text{ABS}}, \quad \mathbf{c}_{\text{acc}} += \tau \alpha \mathbf{c}_{\text{tint}} (\mathbf{c}_{\text{light}} \ell + 0.35), \quad \tau *= (1 - \alpha)$$

當 $\tau < 0.01$ 時提前結束 march。

0.35 這個環境光項可避免完全處於陰影中的煙霧變得完全不可見。最終結果會以 premultiplied alpha 方式(`blendFunc(ONE, ONE_MINUS_SRC_ALPHA)`)疊加到背景上。

5 深度緩衝區整合

5.1 Depth Pre-Pass

在進行 ray marching 之前，`drawModelDepthCapture()` 會將場景中所有圖元渲染到一張附帶 16 位元深度 renderbuffer 的離屏 RGBA half-float FBO (`_modelScreenFBO`) 中。其 shader (`_MODEL_DEPTH_FRAG`) 會將攝影機到表面的歐式距離寫入 *alpha* channel：

$$gl_FragColor = \text{vec4}(0, 0, 0, |\mathbf{w} - \mathbf{eye}|)$$

alpha 為零代表該像素處沒有實體表面。此 pass 會啟用 `DEPTH_TEST(LEQUAL)`、深度寫入與背面剔除，並使用與 ray march 相同的 60° 透視投影與 view matrix，以確保記錄下來的距離具有一致性。

5.2 Ray March 中的整合

在 `drawRayMarch` 中，pre-pass 的 texture 會被綁定為 `uModelBuffer`。對每個像素而言，其 *alpha* 值即為模型深度 d_{model} ，並用以截斷 march 區間：

$$t_{\text{end}} = \min(t_{\text{AABB}}, d_{\text{model}})$$

當 *alpha* 為零時， d_{model} 會被設為 10^9 ，使光線能行進整個 AABB 範圍。若 $t_{\text{end}} \leq t_{\text{start}}$ ，該 fragment 會立即輸出透明黑色，完全省去迴圈的運算成本。

5.3 渲染順序

最終畫面由三個依序執行的 draw call 組成：

1. Depth pre-pass，輸出至離屏 FBO。
2. 光柵化實體幾何，輸出至預設 framebuffer（使用 Blinn-Phong 著色，並開啟 depth test）。
3. Ray march，輸出至預設 framebuffer（讀取 pre-pass 深度，並以 `blendFunc(ONE, ONE_MINUS_SRC_ALPHA)` 疊加到已光柵化的模型上）。

6 互動控制

6.1 第一人稱攝影機

input.js 中的攝影機控制使用 Pointer Lock API 取得原始滑鼠位移量。移動方向的基底由目前的 yaw 推導而來，使得 WASD 移動始終維持在 XZ 平面上，不受 pitch 影響。

按鍵 / 輸入	動作
W/S	前進 / 後退
A/D	左右平移
Shift/Ctrl	上升 / 下降
滑鼠移動（鎖定狀態）	旋轉 yaw 與 pitch
P	暫停 / 繼續模擬
Space	切換深度視覺化顯示

Pitch 被限制在 $\pm 0.44\pi$ 範圍內。移動速度為每次按鍵事件 0.05 個世界單位；滑鼠靈敏度為每像素 0.002 弧度；兩者皆可在 config.js 中設定。

6.2 拋射物系統

在啟動 pointer lock 後按下滑鼠左鍵，會在攝影機前方 0.6 個單位處生成一個邊長 0.25 的黃色方塊，並賦予初始速度 $\mathbf{v}_0 = 8.0 \hat{F} + (0, 1.2, 0)$ 。向上的速度分量讓方塊呈現自然的拋物線軌跡。

每一幀，updateProjectiles(dt) 都會對重力進行積分 ($a_y = -3.0$)，並使用 slab method 將移動路徑與 _colliders 中的每個 AABB 進行相交測試。在發生最近碰撞時，onProjectileHit 會將撞擊點轉換為體積座標 UVW，並呼叫兩次 splat3D——分別注入密度（強度 0.8，半徑 0.002）與溫度（強度 0.2，半徑 0.002）——在接觸點產生煙霧噴發。存活超過 5 秒的拋射物會被自動移除。由於拋射物也會參與 depth pre-pass，因此能正確地遮擋煙霧體積。

6.3 深度視覺化

按下 Space 鍵會切換 config.SHOW_DEPTH_VIZ 旗標。在此模式下，drawDepthViz 會讀取 pre-pass FBO 的 alpha channel，並將攝影機距離映射為灰階值 $(1 - d/20)$ ，即時呈現深度緩衝區的內容以利除錯。

7 限制與未來展望

煙霧無法繞過實體 Depth pre-pass 只是讓煙霧「看起來」在表面處停止，但流體求解器本身並沒有實體邊界條件，速度場與密度場仍會毫無阻礙地穿透幾何體。較完善的解法應是每幀產生一張實體佔用遮罩，並在散度與壓力梯度的計算步驟中強制套用 no-slip 條件。

體積解析度 在 64^3 個體素的解析度下，近距離觀察時會因 trilinear 插值造成的模糊而損失細節。若將線性解析度提升一倍至 128^3 ，則需要 1024×1024 的 atlas（記憶體用量提升 4 倍、shader 運算量約提升 8 倍），在沒有如自適應網格細化等進一步最佳化的情況下，提高解析度的成本相當可觀。

陰影品質 目前僅使用三個固定長度的陰影光線步進，在濃密煙霧中會低估遮蔽程度。增加步進數量，或依據局部密度採用自適應步長，皆能提升精確度，但也會等比例增加 GPU 成本。

單一發射器 預設場景僅配置了一個作用時間為四秒的發射器。然而 EmitterScheduler 類別本身已支援多個同時運作、具備腳本化軌跡的發射器，因此只需少量的程式碼修改即可加入更豐富的煙霧來源。

8 參考文獻

References

- [1] P. Dobrev (PavelDoGreat), *WebGL Fluid Simulation*.
<https://github.com/PavelDoGreat/WebGL-Fluid-Simulation>
- [2] A. Chan, *Real-time Fire Simulation*.
<https://andrewkchan.dev/posts/fire.html>